

Supervising Claude Code — The Manager's Playbook

You are a first-line supervisor of a fast, capable, over-agreeable, memoryless worker. The craft is delegation & verification — not code.

Your instinct is right: this is a management job. But naïvely importing human-team habits will burn you, because this "employee" fails in ways people don't. Below: how to set the delegation dial, how to verify, and where managing Claude departs from managing people.

1 Delegation is a dial, not a switch

The core move is choosing **how close you work** — and changing it mid-task as confidence shifts. Three positions:

- **Pair-program** — you know the domain, review every line, correct constantly. When you're the expert or a mistake is costly.
- **Delegate with a spec** — you define *what + how it's checked*; Claude owns the *how*. Review outcomes, not keystrokes.
- **Delegate with a goal** — unfamiliar domain; you judge results only. The "far-off manager" seat.

Set the dial by three questions: (a) How well-trodden is the domain? (b) How good is your verifier — tests, game outcomes, your own eye? (c) How reversible/costly is a mistake? Strong on all three → delegate to the goal. Weak on all three → pair-program. Most tasks are mixed: **start close, loosen as trust builds within the session.**

2 Match your style to the task, not the tool

A task's "maturity" = *domain-troddenness* × *verifiability*. The same capable agent needs **different management per task**. Your early hacking code was high-maturity (familiar API, testable) — you could delegate freely. Your gang code is low-maturity (novel to you, weak verifier) — it earns close supervision *even though the model is just as competent*. The discomfort you feel isn't the model getting worse; it's you correctly sensing a lower-maturity task.

3 The brief is your highest-leverage act

Output quality tracks **input quality** more than raw model capability. A strong work order carries four things:

- **Context** — the *why*, and where this fits.
- **Constraints** — invariants, what *not* to touch, non-goals.
- **Definition of done** — the observable finish line.
- **The verifier** — how you (or a test) will check it.

Get the plan before the code. The cheapest place to catch a wrong turn is at the brief/plan stage — not in review after 300 lines exist. Approve the *approach* first.

The trap: a vague brief doesn't get questioned, it gets *guessed*. Claude tends to comply rather than push back, so ambiguity you leave in the brief comes back as confident, wrong specifics. Front-load the clarity — it won't be demanded of you.

4 Trust-but-verify — and you never graduate

With human teams, trust *accumulates*: a proven employee earns slack and less oversight. With Claude there is **no persistent worker to build trust in** — each session is a fresh hire carrying the same failure modes as the last. So verification never becomes optional. Two hard rules follow:

- **Never delegate what you can't verify.** If you can't build a check (a test, an in-game metric, your own read), either learn enough to *become* the check, or keep the task yourself.
- **You own the merge.** "The AI wrote it" is not accountability. A manager owns the team's output; you own every line you ship — understood or not.

5 Where this departs from managing people

The failure modes that will actually bite you — because human-management instincts don't cover them:

- No pushback** It tends to agree rather than challenge a flawed premise. **Actively solicit dissent:** "argue against this," "what's the strongest case this is wrong?"
- Confidently wrong** It rarely signals uncertainty the way a junior says "I'm not sure." **Absence of hedging ≠ correctness.**
- No ego or fatigue** Be blunt, iterate endlessly, throw work away freely. Redoing is nearly free — exploit that instead of salvaging a bad first pass.
- Speed asymmetry** It writes faster than you can read. **Review is now your bottleneck** — your core skill shifts from writing to reviewing and verifying.
- Amnesia** Coaching said once is gone next session. Feedback must land in **artifacts you control** (CLAUDE.md, commands, tests) — not in chat.

6 Managing a domain you don't own

When you can't judge the craft (your gang mechanic), judge everything *around* it:

- **Make it teach you enough to be dangerous.** Demand plain-language explanations; if it can't explain simply, distrust it.
- **Triangulate on reasoning.** Ask for the approach *and* the alternatives it rejected *and why*. Reasoning quality is a proxy you can grade without domain depth.
- **Let outcomes be the scoreboard.** The game's numbers (territory %, respect, income) don't lie — wire them as your verifier.
- **Shrink the batch.** Small increments you can observe beat big drops you can't.

7 Run retros on the system, not the task

Periodically ask: *what keeps going wrong?* Recurring failures are bugs in your **management system**, not one-off mistakes — and since the agent carries no memory, you fix them where they persist: tighten CLAUDE.md, add a slash command, add a test or hook. You improve the "team" only by improving the artifacts around it.

Manager's operating rules

1. Set the delegation dial **deliberately**; move it as confidence changes.
2. **Never delegate what you can't verify.**
3. The brief carries the load — context, constraints, definition of done, verifier.
4. **Plan before code.**
5. Solicit dissent; absence of hedging isn't correctness.
6. **You own the merge** — every line, understood or not.
7. Small batches, frequent checks.
8. Bank lessons in **artifacts** (CLAUDE.md / commands / tests), not in chat.